Student 4 **ANUMULA VS SANTHOSH RADHA KRISHNA MURTHY** 192312043RA 4 / 4 submitted

Q1. Arrays, Array Applications and Structures

CODE

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[1000];
    int count[256] = {0};
    scanf("%[^\n]", str);
    int len = strlen(str);
    for (int i = 0; i < len; i++) {
        count[(unsigned char)str[i]]++;
    }
    int found = 0;
    for (int i = 0; i < len; i++) {
        if (count[(unsigned char)str[i]] == 1) {
            printf("First non-repeating character: %c\n", str[i]);
            found = 1;
            break;
        }
    }
    if (!found) {
        printf("No non-repeating character found\n");
    }
    return 0;
}
```

INPUT

```
b a r b e r
```

OUTPUT

```
(no output)
```

Q2. Arrays, Array Applications and Structures

CODE

```
#include <stdio.h>
int searchRotated(int a[], int n, int target) {
    int low = 0, high = n - 1;
    while (low ≤ high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[low] ≤ a[mid]) {
            if (target ≥ a[low] && target < a[mid])
                high = mid - 1;
            else
                low = mid + 1;
        }
        else {
            if (target > a[mid] && target ≤ a[high])
                low = mid + 1;
            else
                high = mid - 1;
        }
    }
    return -1;
}
int main() {
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    int target;
    scanf("%d", &target);
    int idx = searchRotated(a, n, target);
    if (idx ≠ -1)
        printf("Found at index %d\n", idx);
    else
        printf("Not found\n");
    return 0;
}
```

INPUT

```
7
4 5 6 7 0 1 2
0
```

OUTPUT

```
(no output)
```

Q3. Arrays, Array Applications and Structures

CODE

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int start, end;
} Interval;
int compare(const void *a, const void *b) {
    Interval *x = (Interval *)a;
    Interval *y = (Interval *)b;
    if (x->start != y->start)
        return x->start - y->start;
    return x->end - y->end;
}
int main() {
    int n;
    scanf("%d", &n);
    Interval *arr = (Interval *)malloc(n * sizeof(Interval));
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &arr[i].start, &arr[i].end);
    }
    qsort(arr, n, sizeof(Interval), compare);
    Interval *merged = (Interval *)malloc(n * sizeof(Interval));
    int m = 0;
    for (int i = 0; i < n; i++) {
        if (m == 0) {
            merged[m++] = arr[i];
        } else {
            if (arr[i].start <= merged[m - 1].end) {
                if (arr[i].end > merged[m - 1].end)
                    merged[m - 1].end = arr[i].end;
            } else {
                merged[m++] = arr[i];
            }
        }
    }
    for (int i = 0; i < m; i++) {
        printf("%d %d", merged[i].start, merged[i].end);
        if (i != m - 1) printf("\n");
    }
    free(arr);
    free(merged);
    return 0;
}
```

INPUT

```
5
1 3
2 4
6 8
7 10
12 15
```

OUTPUT

```
(no output)
```

Q4. Arrays, Array Applications and Structures

CODE

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int freq[256] = {0};
    scanf("%99s", str);
    int len = strlen(str);
    for (int i = 0; i < len; i++) {
        freq[(unsigned char)str[i]]++;
    }
    printf("Rearranged string: ");
    for (int f = len; f ≥ 1; f--) {
        for (int c = 0; c < 256; c++) {
            if (freq[c] == f) {
                for (int k = 0; k < f; k++) {
                    printf("%c", c);
                }
            }
        }
    }
    printf("\n");
    return 0;
}
```

INPUT

```
tree
```

OUTPUT

```
(no output)
```